

SYSTEM DOCUMENTATION

RepoSage

Complete System Documentation

Generated 2026-10-07

2

EXECUTION ENGINES

9

AUDIT SCANNERS

10

FRONTEND PAGES

2

LLM PROVIDERS

RepoSage is a full-stack autonomous repository assistant: it searches GitHub for a repo matching a described task, analyzes its structure, generates architecture diagrams, answers questions about it via retrieval-augmented chat, attempts the task through a self-healing LLM execution loop, audits code quality with nine static scanners, and detects copy-pasted or duplicated code across repos.

Frontend: React + Vite on Vercel · Backend: FastAPI on Railway (Docker)

Contents

Contents

2

1. Introduction

4

2. System Architecture at a Glance

4

2.1 Layers

4

2.2 Two execution systems — the most important thing to understand

4

3. Frontend — React + Vite SPA (Vercel)

4

3.1 Pages

5

3.2 The API client (api.js)

5

4. Backend — FastAPI on Railway (Docker)

5

4.1 Auth & History

5

4.2 Semantic Search

6

4.3 Repository Analysis (analyzer.py)

6

4.4 Architecture Diagram Generation (architect.py)

6

4.5 RAG Chat (chat.py + rag.py)

7

4.6 Self-Healing Task Executor (executor.py)

7

4.7 Audit Engine — 9 Scanners (tester.py)

7

4.8 Copy / Duplicate Detection

7

4.9 LLM Gateway (llm.py)

8

5. repo-task executor — the Separate Agentic Pipeline

8

5.1 Orchestrator

8

5.2 Graph Engine & Smart Context

8

5.3 Claude Code Agent

8

5.4 Sandbox Tester

9

5.5 PR & Docs Writer

9

5.6 Standalone API server (dev-only)

9

6. Two Execution Systems, Side by Side

9

7. Data Stores & External Services

9

8. Deployment & Infrastructure	10
8.1 Split deployment: Vercel + Railway	10
8.2 Memory & CPU tuning on a constrained container	10
9. Complete API Route Reference	10
9.1 Auth & History	10
9.2 Search, Analyze, Chat, Architecture	11
9.3 Execution & Output	11
9.4 Audit & Copy Detection	11
10. Appendix — Key Environment Variables	12

1. Introduction

RepoSage is a full-stack system that takes a GitHub repository and a natural-language task, and carries that task through an entire pipeline: finding the right repository, understanding its structure, explaining its architecture, answering questions about it, attempting the task autonomously, checking the resulting code quality, and catching duplicated or copy-pasted code. This document explains, feature by feature, how every part of it actually works.

The system has two halves that deploy and scale independently: a [React + Vite](#) single-page frontend served from Vercel, and a [FastAPI](#) backend running in a Docker container on Railway. A third, fully separate subsystem — the **repo-task executor** — lives inside the same backend image but runs as an independent agentic pipeline with its own LLM provider (Anthropic Claude, instead of the backend's own Groq-based models).

Three different techniques do the heavy lifting in different places: **deterministic static analysis** (the 9-scanner audit engine, the AST/graph-based repo analyzer), **embedding-based retrieval** (semantic search, RAG chat, copy detection — bi-encoders and FAISS), and **LLM reasoning** (task execution, architecture diagrams, chat answers, patch generation — Groq or Claude, depending on the feature). Knowing which technique is behind a given feature is the key to understanding why it behaves the way it does — this document calls that out for every section.

2. System Architecture at a Glance

Top to bottom: a user's browser talks to the Vercel-hosted frontend over HTTPS; the frontend talks to the Railway-hosted backend over REST and Server-Sent Events (SSE) for anything long-running. The backend owns nine feature modules. One of them — the Self-Healing Executor — can also hand a task off to the separate repo-task executor pipeline as a subprocess. Both the backend and the repo-task executor talk out to MongoDB, the filesystem, GitHub, and their respective LLM providers.

2.1 Layers

- **Client** — the user's browser.
- **Frontend (Vercel)** — React + Vite SPA, 10 pages, talks to the backend only through [frontend/src/api.js](#).
- **Backend (Railway, Docker)** — FastAPI app ([backend/main.py](#)), 9 feature modules, all behind JWT auth where relevant.
- **repo-task executor** — a separate Python package, invoked by the backend as a subprocess, not imported as a library.
- **Data & external services** — MongoDB, the job-outputs filesystem volume, GitHub's API, Groq's API, and Anthropic's API.

2.2 Two execution systems — the most important thing to understand

RepoSage can attempt a user's task in **two independent ways**, and they are easy to confuse because both appear in the UI as an “Execute” step:

- [/api/execute](#) runs [backend/executor.py](#) in-process. Groq-driven. Up to 7 retries. Shallow, format-only output validation.
- [/api/taskexec](#) shells out to the repo-task executor's [run_task.py](#) as a subprocess. Claude-driven (claude-haiku-4-5). Builds a graph-based “smart context” before every edit, and can open a real GitHub pull request.

Section 6 compares these two systems side by side in detail.

3. Frontend — React + Vite SPA (Vercel)

The frontend is a single-page app with 10 route-level pages, each owning one step of the pipeline. It never talks to any backend route directly with raw `fetch` — every call goes through `frontend/src/api.js`, a single client module.

3.1 Pages

Page	What it does
LandingPage	Marketing / entry screen. No API calls.
SearchPage	Describe a task (optionally upload files) and search GitHub for matching repos.
SelectPage	Browse / re-search candidates and pick one to work with.
AnalyzePage	Shows structural analysis, lets you chat with the repo, and build/check its RAG index.
ArchitectPage	Renders LLM-generated Mermaid diagrams (architecture / sequence / dataflow).
ExecutePage	Drives the Groq-based self-healing executor loop; handles mid-run credential prompts.
TaskExecPage	Drives the separate, Claude-based repo-task executor pipeline.
TestPage	Runs the 9-scanner audit engine and renders / downloads the PDF report.
OutputPage	Lists, downloads and deletes a job's output artifacts.
CopyDetectPage	Three-mode duplicate detector: cross-repo, self-scan, corpus search.
HistoryPage	Lists the signed-in user's past task-history entries.

3.2 The API client (api.js)

Base URL: `const BASE = (import.meta.env.VITE_API_URL || '') + '/api'`. Behind the single-origin Docker deploy, a relative path works because Caddy proxies `/api` to the backend. Split across Vercel and Railway, `VITE_API_URL` must point at the Railway backend's own URL — there is no same-origin proxy across two different platforms.

- **Auth token** kept in `localStorage['rm_token']`; `apiFetch()` attaches it as a Bearer header automatically.
- **Session expiry** a 401 on a non-auth endpoint clears the token, reloads the page, and throws “Session expired”; a 401 on login/register is shown as a real credential error instead.
- **Streaming (SSE)** four long-running features (execute, taskexec, test, architect/rag/copydetect/corpus) use a hand-rolled fetch + ReadableStream reader that manually parses `data:` lines, deliberately not the browser's `EventSource` API — `EventSource` cannot send custom headers, so it cannot carry the auth token.

Deployment: static build served from Vercel's edge network; see Section 8 for the Vercel/Railway split details.

4. Backend — FastAPI on Railway (Docker)

The backend is one FastAPI app (`backend/main.py`) exposing roughly 30 routes across 9 feature areas. It runs as a single Docker container on Railway, with the repo-task executor package copied into the same image at `/opt/repo-task-executor`.

4.1 Auth & History

Passwords go through a SHA-256 pre-hash (so `bcrypt`, which silently truncates at 72 bytes, never sees a long password) and then `bcrypt` itself, run in a dedicated 4-worker thread pool so hashing never blocks the `asyncio` event loop. JWTs are HS256, carry `{sub, exp, iat}`, and expire after 7 days.

- **Registration / login** `create_user` / `authenticate_user` in `backend/auth.py`; login failures are deliberately generic (“Invalid email or password” either way) to avoid revealing which field was wrong.
- **Password reset** a `secrets.token_urlsafe(32)` token, 1-hour expiry, emailed via Gmail SMTP; an unknown email returns a silent empty response rather than an error, so the endpoint can't be used to enumerate registered emails.
- **History** one MongoDB collection (`history`), scoped to `{_id, user_id}` on every query so cross-user access is structurally impossible; every `history_create` call prunes down to the newest 20 entries per user.

4.2 Semantic Search

Scores GitHub search candidates against the user's task with a `cross-encoder/ms-marco-MiniLM-L-6-v2` cross-encoder as the primary path — it scores every (task, repo) pair jointly, which is far more accurate than comparing separately-encoded vectors. Because the candidate set from GitHub is small (a few dozen repos, not millions), the usually-expensive cross-encoder pass is actually cheap enough to be the default; an `all-MiniLM-L6-v2` bi-encoder (cosine similarity over separately-encoded vectors) is kept only as a fallback if the cross-encoder fails to load.

This is the inverse of the textbook retrieve-then-rerank order — deliberately, because the candidate set here is already small.

4.3 Repository Analysis (`analyzer.py`)

Builds a real, structural picture of a repository — not an LLM summary of it. The pipeline: shallow-clone → parse every Python file and notebook into an AST → build a symbol table → build a **module dependency graph** and a **function/class call graph** → score every module and class → detect clusters and import cycles → run exactly one LLM call at the very end to add docstrings, a readme summary and a task plan on top of the already-computed structure.

- **Module dependency graph (MDG)** built primarily by **actually running** the repo's imports through a custom `sys.meta_path` hook that traces real `importlib` loads — far more accurate than guessing from import statements alone — with a pure-AST fallback if runtime tracing produces nothing.
- **Function call graph (FCG)** resolves call targets through the symbol table plus per-file import-alias maps, including `self.attr()` calls resolved via a type map scraped from `__init__`, and `super()` calls attributed to base classes.
- **Module/class scoring** a weighted formula over lines of code, class count, function count, import count, and inbound/outbound graph edges; cyclomatic complexity and Halstead metrics come from `radon`.
- **Clustering & cycles** real community detection (`networkx.greedy_modularity_communities`) per connected component; import cycles found via Kosaraju's strongly-connected-components algorithm on the MDG.
- **The one LLM call** `_llm_enrich()` adds docstrings, a readme summary, a 5-step task plan, the entry point and run command — it never alters the structural data the rest of the pipeline computed.

4.4 Architecture Diagram Generation (`architect.py`)

Unlike the analyzer's static graphs, this feature is **entirely LLM-driven**: it fetches the repo's file tree and README from the GitHub REST API, asks the LLM for a free-text explanation, then asks a second time for strict JSON describing nodes and edges (retried up to 3 times on a parse or validation failure), and compiles that JSON into a Mermaid diagram.

- **Three diagram kinds** `architecture` (component flowchart), `sequence` (one end-to-end runtime flow), `dataflow` (external/process/store flow) — listed by `GET /api/architect/kinds` so the UI never offers a kind that doesn't exist.
- **Streamed as SSE** the final event carries the compiled Mermaid source, the explanation, the raw graph JSON, and which model in the Groq chain actually answered.

4.5 RAG Chat (chat.py + rag.py)

A second, separate embedding index from semantic search — one FAISS index per repository, built with LangChain and a local, CPU-only `all-MiniLM-L6-v2` embedder, stored at `~/.repomaster/rag/<repo>/`. Building an index clones the repo, loads allowed file types (skipping lockfiles and virtual environments), splits them with language-aware chunking, embeds, and saves the index.

Answering a question combines **two** sources of context: the analyzer's already-computed structural JSON (readme summary, entry point, modules, classes, import cycles, task plan) is always injected into the system prompt, and the top-6 RAG chunks for the specific question are retrieved and prepended as “primary evidence” whenever an index exists for that repo.

4.6 Self-Healing Task Executor (executor.py)

The Groq-driven execution engine behind `/api/execute`. Before attempting anything, an upfront LLM classification can flag a task as categorically impossible in this environment (e.g. it needs a GPU, or a paid third-party API key the user hasn't supplied) — that short-circuits straight to a `not_runnable` result without ever entering the retry loop.

- **Retry loop** up to `MAX_RETRIES = 7` attempts (env-overridable via `EXECUTOR_MAX_RETRIES`); each failed attempt's error feeds back into the next attempt's prompt.
- **Exhausted-retries path** a genuine failure path exists (`if attempt >= MAX_RETRIES - 1: break`), followed by one deterministic, non-LLM pattern-based auto-fix attempt as a last resort.
- **Output validation is shallow** the `_FORMAT_CHECKS` table only validates a handful of binary formats by magic bytes (PNG/JPEG/PDF/ZIP/XLSX/DOCX/SVG/HTML/JSON); most other file types only get an existence/non-empty check — meaning the model has considerable room to produce output that passes without being semantically correct. This was directly observed in this project's own testing.

4.7 Audit Engine — 9 Scanners (tester.py)

A purely deterministic static-analysis pipeline — confirmed to contain zero LLM calls anywhere. Nine scanners run against the target repo: 7 wrap an established open-source tool, 2 are RepoSage's own graph-based analyses.

- **Wrapped tools** `ruff` (lint), `bandit` (security), `pip-audit` (dependency CVEs), `mypy` (types), `detect-secrets`, `vulture` (dead code), `semgrep` (patterns).
- **RepoSage's own: Architecture Health** reuses the analyzer's module/function graphs to flag graph-shaped defects (god modules, import cycles) that no single-file linter can see.
- **RepoSage's own: Exception Flow** an interprocedural exception-propagation graph, solved as a fixpoint, that flags unhandled/over-broad exception handling across function and module boundaries — something no per-file Python linter can reason about.
- **Scoring** CVSS-weighted finding density per thousand lines of source, decayed exponentially into a 0–100 score and an A–F grade; tests/docs/examples are reported in full but excluded from the grade.
- **Output** the same engine that powers the in-app audit tab also generates the professional, cover-page PDF report available for download from the Test page.

On the memory-capped Railway deployment, the audit engine explicitly frees the search reranker's memory before running, and runs semgrep single-threaded — see Section 8.

4.8 Copy / Duplicate Detection

Three independent detectors live behind one feature, each solving a different problem at a different scale:

- **Cross-repo (copydetector/)** a vendored k-gram fingerprinting library (winnowing, k=25) that does literal/token-level diffing between two specific repos — finds code that was copy-pasted, even with variable names changed.

- **Self-scan (dupdetect.py)** a semantic near-duplicate-function detector using the same `all-MiniLM-L6-v2` bi-encoder as RAG chat, scoped to same-class/module pairs within one repo, threshold 0.68 cosine similarity.
- **Corpus search (corpus.py)** a single, growing FAISS index of function embeddings across many repos a user has added — bi-encoder only, no cross-encoder, because the corpus can grow to thousands of functions and a cross-encoder doesn't scale to that.

4.9 LLM Gateway (llm.py)

Every Groq-backed feature (search's cross-encoder fallback aside) goes through one shared chat function with built-in multi-key, multi-model fallback. Keys and models are re-read from the environment on every single call, so an `.env` edit or a Railway variable change takes effect without a code deploy.

- **Model chain** default `openai/gpt-oss-120b → 20b → qwen3.8-27b → gpt-oss-safeguard-20b`, overridable via `GROQ_MODELS`; iterates models on the outside, keys (`GROQ_API_KEY_1..4`) on the inside.
- **Smart failure handling** an HTTP error is classified into a model-is-gone case (skip straight to the next model, don't waste the other 3 keys on it) versus a malformed-request case (fail immediately, don't retry at all) versus a generic transient error (try the next key).
- **Token-budget exhaustion** explicitly detected and treated as a real failure, not a silent success — a reasoning model can spend its entire token budget “thinking” and return empty content, which previously surfaced as confusing downstream errors like the analyzer's “No JSON found in: ”.

5. repo-task executor — the Separate Agentic Pipeline

A second, independent Python package, not imported by the main backend as a library — the backend shells out to its `run_task.py` as a subprocess and streams its stdout back to the frontend as SSE, scrubbing anything that looks like a secret along the way. It uses **Anthropic's Claude** (claude-haiku-4-5) rather than Groq, and is built around a much more deliberate “read the repo like a graph, edit only what's relevant” philosophy.

5.1 Orchestrator

Runs its own self-healing retry loop, structurally similar to the backend's executor but independent of it. A `touched[]` file accumulator persists across attempts on disk — without it, an attempt that fixes the bug followed by an attempt that (correctly) sees green tests and returns an empty patch would report “no file changes were needed” even though real changes were made earlier.

5.2 Graph Engine & Smart Context

Before any edit is attempted, three graphs are built over the whole repo: a **Hierarchical Code Tree** (HCT, directory structure with relevance scoring), the **Function Call Graph** (FCG), and the **Module Dependency Graph** (MDG, with PageRank-based risk scoring). A context builder then picks the most task-relevant files, expands outward to their direct callers/callees as “impact” context, and — for any file over 150 lines — extracts only the specific function bodies relevant to the task (via AST line-range extraction) plus a few lines of surrounding context, instead of sending whole files.

- **Token budget** the whole assembled context is capped around 200,000 characters (roughly 60k tokens), so even a large repo stays inside what Claude can usefully read in one call.
- **Why it matters** this is what lets a small, fast model operate correctly on a large unfamiliar codebase — it edits a surgical subset instead of being shown (or guessing at) the whole repo.

5.3 Claude Code Agent

Calls the Anthropic API directly (`claude-haiku-4-5` by default) and asks for changes in a patch-hunk format, which the writer then applies to disk.

5.4 Sandbox Tester

Runs the repo's existing test suite inside a Docker sandbox when Docker is available and reachable, falling back to direct subprocess execution otherwise. Critically, it only looks for test files by filename pattern (`test_*.py`, `*_test.py`, `spec.js`); if none exist in the repo, the test step is skipped entirely and the result is auto-marked as passed — meaning a repo with no pre-existing tests gets exactly one, unverified attempt, never a real retry loop, because there is nothing to check pass/fail against.

5.5 PR & Docs Writer

Commits the changes to a new branch and opens a real GitHub pull request via the GitHub API (or commit-only, for simple tasks). All git/GitHub output is passed through a secret-scrubbing filter before it reaches any log or the UI — added after a real incident where a GitHub token appeared verbatim in a failed git command's output. A separate documentation agent can also generate docs for the changed code using the same Claude client and the same AST-based code reading.

5.6 Standalone API server (dev-only)

The package also ships its own FastAPI app (`api/server.py`) with `/run`, `/job/{id}`, `/jobs` and `/analyze` routes and an in-memory job store. This is **not** what the production backend uses — the production path is the subprocess/SSE bridge described above. This server appears to exist as a self-contained way to run or demo the repo-task executor package on its own.

6. Two Execution Systems, Side by Side

Route	<code>/api/execute</code>	<code>/api/taskexec</code>
Engine	backend/executor.py (in-process)	repo-task executor (subprocess)
LLM provider	Groq (multi-key, multi-model)	Anthropic Claude (claude-haiku-4-5)
Context strategy	LLM reasons over task + repo info directly	Graph-based smart context (HCT/FCG/MDG)
Retries	up to 7, upfront not-runnable short-circuit	self-healing loop, touched[] accumulator
Test verification	format-only artifact validation	runs repo's own tests if present, else auto-pass
Can open a PR	No	Yes (or commit-only mode)

Both systems share one structural weakness worth knowing about: success is only as meaningful as the verification behind it. `/api/execute`'s format-only checks and `/api/taskexec`'s no-pre-existing-tests auto-pass both mean a task can be reported as “done” without its output being semantically verified, whenever the repo or the task doesn't give the system anything concrete to check against.

7. Data Stores & External Services

- **MongoDB** single database (`repomaster`), collections for users and task history; accessed via a lazily-created, pooled `AsyncIOMotorClient`.
- **Job Outputs** per-job artifacts written to a filesystem volume, served and deleted through `/api/outputs/{job_id}`.
- **GitHub API** repo search, file-tree/README fetching, cloning, and — from the repo-task executor — opening real pull requests.
- **Groq API** the backend's LLM provider for search reranking, analysis enrichment, architecture diagrams, RAG chat answers, and the self-healing executor.

- **Anthropic API** used exclusively by the repo-task executor's Claude Code Agent and doc generator — a second, independent LLM provider from the rest of the system.
- **Embedding models (offline)** `all-MiniLM-L6-v2` and `ms-marco-MiniLM-L-6-v2` are baked into the Docker image and loaded with `HF_HUB_OFFLINE=1`, so no live call to huggingface.co happens at runtime.

8. Deployment & Infrastructure

8.1 Split deployment: Vercel + Railway

The frontend (static Vite build) deploys to Vercel; the backend (Docker image built from `docker/Dockerfile.backend`, which also copies in the repo-task executor package) deploys to Railway. Because the two platforms don't share an origin, `VITE_API_URL` on the frontend must point at the Railway backend's public URL, and the backend's CORS configuration must allow the Vercel origin. A single-origin alternative also exists (`docker/Caddyfile`, `docker-compose.yml`) where Caddy proxies `/api` to the backend from the same origin.

8.2 Memory & CPU tuning on a constrained container

Running a reranker model, an audit engine that spins up per-job virtual environments, and a PyTorch-based embedder all in one process, inside a memory- and CPU-capped container, needed several deliberate fixes:

- **Reranker memory release** the audit engine explicitly frees the resident cross-encoder (`release_reranker()`) before it starts, so an earlier search in the same process doesn't leave no headroom for the audit's own memory spike.
- **Per-scanner garbage collection** the audit loop runs `gc.collect()` after each of the 9 scanners, rather than waiting for the next allocation to trigger it.
- **Single-threaded semgrep** `SEMGREP_JOBS=1` trades scan speed for a much smaller peak-memory footprint on the capped plan.
- **CPU thread oversubscription** `OMP_NUM_THREADS` / `MKL_NUM_THREADS` are pinned to the container's real vCPU allocation — PyTorch otherwise spawns threads based on the host's full core count, not the cgroup-enforced quota, which was observed to throttle reranker batches from sub-second to 12–31 seconds each.
- **Offline model loading** `HF_HUB_OFFLINE=1` / `TRANSFORMERS_OFFLINE=1` stop the Hugging Face Hub client from making live network calls to check for model updates on every load, even though the model is already baked into the image.

9. Complete API Route Reference

Every route exposed by `backend/main.py`, grouped by feature area.

9.1 Auth & History

Endpoint	What it does
POST <code>/api/auth/register</code>	Create an account.
POST <code>/api/auth/login</code>	Sign in, returns a JWT.
POST <code>/api/auth/forgot-password</code>	Email a password-reset link.
POST <code>/api/auth/reset-password</code>	Consume the reset token.
GET <code>/api/auth/me</code>	Current signed-in user.
GET <code>/api/history</code>	List this user's task-history entries.

Endpoint	What it does
GET /api/history/{id}	One history entry.
POST /api/history	Create a history entry.
PATCH /api/history/{id}	Update a history entry.
DELETE /api/history/{id}	Delete a history entry.

9.2 Search, Analyze, Chat, Architecture

Endpoint	What it does
POST /api/upload	Upload files alongside a search.
POST /api/search	Semantic repo search (cross-encoder).
POST /api/analyze	Structural repository analysis.
POST /api/chat	RAG-grounded repo Q&A.
POST /api/rag/build	Build a repo's RAG index (SSE).
GET /api/rag/status/{owner}/{name}	RAG index status.
GET /api/architect/kinds	Available diagram kinds.
POST /api/architect	Generate an architecture diagram (SSE).

9.3 Execution & Output

Endpoint	What it does
POST /api/execute	Groq self-healing executor (SSE).
POST /api/taskexec	Claude repo-task executor (SSE).
GET /api/outputs/{job_id}	List a job's output files.
GET /api/outputs/{job_id}/{filename}	Download one output file.
DELETE /api/outputs/{job_id}	Delete a job's outputs.
POST /api/credentials/{job_id}	Submit a credential the agent asked for.

9.4 Audit & Copy Detection

Endpoint	What it does
POST /api/test	9-scanner audit engine (SSE).
POST /api/copydetect	Cross-repo / self-scan copy detection (SSE).
GET /api/corpus/status	Corpus index status.
POST /api/corpus/add	Add a repo's functions to the corpus (SSE).
POST /api/corpus/search	Search the corpus (SSE).

10. Appendix — Key Environment Variables

Variable	What it does
JWT_SECRET	Signing key for auth tokens.
MONGO_URL	MongoDB connection string.
GROQ_API_KEY_1..4	Backend LLM provider keys (rotated).
GROQ_MODELS / GROQ_MODEL	Override the Groq model fallback chain.
ANTHROPIC_API_KEY	repo-task executor's Claude provider key.
CLAUDE_MODEL	Override the repo-task executor's model.
VITE_API_URL	Frontend → backend base URL (split deploys).
EXECUTOR_MAX_RETRIES	Override the self-healing executor's retry count.
SEMGREP_JOBS	Cap semgrep's parallel worker count.
OMP_NUM_THREADS / MKL_NUM_THREADS	Pin PyTorch/BLAS threads to the real vCPU quota.
HF_HUB_OFFLINE / TRANSFORMERS_OFFLINE	Stop live Hugging Face Hub network calls.

End of document. Generated from a direct reading of the RepoSage codebase.